

FUNDATEC

Procesamiento Digital de Señales

Profesor:
Michael Grüner

Estudiante:
Noel Pérez Cáceres

III Cuatrimestre 2026

Tarea 2

Solución Tarea 2

1. Lectura, inspección y escritura de archivos WAV

Investigue cómo realizar las siguientes operaciones con archivos WAV en Python. Implemente todas ellas y entregue el código fuente en el repositorio. En el reporte, describa los conceptos y procedimientos utilizados.

- a) Investigue cómo leer un archivo WAV e implemente la lectura.

La lectura se realiza a través de `scipy.io.wavfile.read`, este devuelve `(fs, datos)`. Lee el PCM y float; existe un arreglo llamado *datos* el cual tiene la forma `(N,)` o `(N, canales)`.

- b) Investigue cómo averiguar la frecuencia de muestreo y cómo calcular, en segundos, la duración del archivo usando el número de muestras y la frecuencia de muestreo. Implemente ambos cálculos usando la frecuencia de muestreo del archivo.

Frecuencia y duración: `fs` es el número de muestras por segundo (Hz). La duración es $T = N / fs$ [s], con `N` muestras por canal (`datos.shape[0]`).

- c) Investigue cómo averiguar el ancho de muestra. Describa qué significa, cuáles anchos y formatos de muestra soporta WAV y cuáles son sus rangos representables. A partir de ello, explique cómo convertir las muestras a punto flotante en el rango de -1 a 1 e implemente esa conversión.

Ancho de muestra: es el número de bits por muestra y fija el rango representable.

Formato	Rango	Normalización a $[-1, 1]$
PCM 8 bits (sin signo)	$0 \dots 255$	$(x - 128) / 128$
PCM 16 bits	$-2^{15} \dots 2^{15}-1$	$x / 2^{15}$
PCM 24 bits	$-2^{23} \dots 2^{23}-1$	$x / 2^{23}$
PCM 32 bits	$-2^{31} \dots 2^{31}-1$	$x / 2^{31}$
Float 32/64	nominal $[-1, 1]$	sin cambio

- d) Investigue qué representa el número de canales y cómo averiguarlo en un archivo WAV. Implemente la inspección de los canales.

Canales: es la cantidad de señales intercaladas por instante (1 = mono, 2 = estéreo). Se obtiene de `datos.ndim` y `datos.shape[1]`; en el encabezado WAV es el campo `NumChannels`.

- e) Investigue cómo escribir un archivo WAV. Escoja un ancho de muestra distinto al del archivo original, desnormalice las muestras del rango de $[-1, 1]$ al rango representable por ese nuevo ancho de muestra y escriba un nuevo archivo WAV con ellas. Implemente esta operación.

Escritura: se desnormaliza con saturación (clip) al rango del nuevo ancho, por ejemplo $\text{round}(x \cdot (2^{15}-1))$ para 16 bits, y se escribe con `wavfile.write`. El script usa 32 bits si el original es 16, y 16 bits en el resto de los casos.

2. Convolución de una señal con una respuesta al impulso

Se proveen una señal de audio `señal.wav` y una respuesta al impulso `respuesta_al_impulso.npy`. Implemente un script de Python que realice los requerimientos especificados en los siguientes ítems. Entregue el script en el repositorio y describa en el reporte el procedimiento y los resultados.

- a) Cargue la señal de audio y la respuesta al impulso.

Carga: el audio se lee con `wav_io` y se normaliza a $[-1, 1]$; la respuesta al impulso con `np.load`

- b) Convolucione ambas señales utilizando `numpy.convolve`. Puede utilizar otra implementación de convolución existente o implementar una propia. No utilice `scipy.signal.filter` ni `scipy.signal.lfilter`, ni variantes optimizadas para filtrado.

Convolución: $y(n) = \sum_k x(k) \cdot h(n-k)$ con `numpy.convolve`, aplicada a cada canal si el audio es estéreo.

- c) Determine y justifique la longitud teórica de la señal resultante a partir de las longitudes de la señal de audio y de la respuesta al impulso. Compruebe que coincide con la longitud obtenida.

Longitud teórica: $N_y = N_x + N_h - 1$.

Si x va de k_i a k_f y h de l_i a l_f , la salida va de $k_i + l_i$ a $k_f + l_f$.

Entonces $N_y = (k_f + l_f) - (k_i + l_i) + 1 = N_x + N_h - 1$.

El script imprime la longitud teórica, la obtenida y si coinciden.

- d) Escriba la señal resultante en un archivo WAV e inclúyalo en el repositorio. Compare el audio antes y después de la convolución y describa el efecto que percibe del sistema.

Escritura de la señal: se guarda en 16 bits. Si el pico supera 1.0 se normaliza por el pico para evitar saturación, y el script lo avisa. Ese factor debe mencionarse en el reporte.

3. Generación de un eco mediante convolución

Utilice su entendimiento de la convolución y de la respuesta al impulso para resolver los ítems siguientes. Utilice únicamente la materia vista hasta el momento en clase; no

está permitido utilizar conceptos que no se hayan estudiado hasta el momento.

a) Diseñe una respuesta al impulso que genere una réplica de la señal original retrasada N segundos y con amplitud relativa A . Explique su funcionamiento.

- Con $D = \text{round}(N \cdot f_s)$ muestras de retraso:
 - $h(n) = \delta(n) + A \cdot \delta(n - D)$
- Funcionamiento:
 - Por la descomposición en impulsos, $x(n) = \sum x(k) \delta(n-k)$, y como el sistema es LTI, la salida es $y(n) = \sum x(k) h(n-k)$.
 - Con esta h , $y(n) = x(n) + A \cdot x(n-D)$.
 - El primer impulso deja pasar la señal original. El segundo aporta una copia retrasada N segundos y escalada por A .
- Longitud de la salida: $N_x + D$ muestras, es decir $N_x + N_h - 1$ con $N_h = D + 1$. El audio dura N segundos más.
- Si $A = 0$ el sistema es la identidad, y si $D = 0$ queda $h(0) = 1 + A$.

b) Implemente lo anterior como un script de Python. El script debe recibir como argumentos un archivo WAV de entrada, la amplitud relativa del eco (número de punto flotante entre 0 y 1, inclusive) y el retraso en segundos (número de punto flotante). Escriba la señal resultante en un archivo WAV con el mismo nombre que el archivo de entrada y el sufijo `_out` antes de la extensión. Por ejemplo, el archivo `archivo.wav` debe producir `archivo_out.wav`.

Valida que $0 \leq A \leq 1$ y que el retraso sea ≥ 0 .

Calcula D con la f_s del archivo.

La salida se escribe con el sufijo `_out`.

c) Extienda el script para que admita y procese archivos WAV con cualquier formato de muestra y cualquier número de canales. El archivo de salida debe conservar el formato de muestra y el número de canales del archivo de entrada.

`soundfile` lee a float en $[-1, 1]$, procesa cada canal por separado y escribe con el mismo subtype (bits) y número de canales. Esto cubre PCM de 8/16/24/32 bits y float.

4. Clasificación de sistemas discretos

Para cada uno de los siguientes sistemas, clasifíquelo según las propiedades indicadas. Justifique cada clasificación aplicando la definición correspondiente.

- Lineal o no lineal.
- Invariante o variante en el tiempo.
- Estático o dinámico.
- Causal o no causal.

- Estable o inestable.

a) $y(n) = x(n) \cos(\pi n) = (-1)^n x(n)$

Lineal: $T[a_1 x_1 + a_2 x_2] = (a_1 x_1 + a_2 x_2)(-1)^n = a_1 T[x_1] + a_2 T[x_2]$.

Variante: $y_k(n) = x(n - k)(-1)^n$

, pero $y(n - k) = x(n - k)(-1)^{n-k}$

; difieren cuando k es impar.

Estático: solo usa $x(n)$. Causal: no usa futuro. Estable: $|y(n)| = |x(n)| \leq M_x$.

b) $y(n) = x(n - 2) + x(n + 1)$

Lineal: es suma ponderada de versiones desplazadas de x .

Invariante: $y_k(n) = x(n - k - 2) + x(n - k + 1) = y(n - k)$.

Dinámico: usa $x(n - 2)$ y $x(n + 1)$, no $x(n)$. No causal: depende de $x(n + 1)$.

Estable: $|y(n)| \leq 2M_x$.

c) $y(n) = x(2n)$

Lineal: $T[a_1 x_1 + a_2 x_2](n) = a_1 x_1(2n) + a_2 x_2(2n)$.

Variante: $y_k(n) = x(2n - k)$, pero $y(n - k) = x(2n - 2k)$; para $k = 1$ resultan $x(2n - 1) \neq x(2n - 2)$

en general.

Dinámico: $y(1) = x(2)$. No causal: $y(1) = x(2)$ usa una muestra futura.

Estable: $|y(n)| = |x(2n)| \leq M_x$.

d) $y(n) = x(n)u(n)$

Lineal: $u(n)$ es un factor jo, y $T[a_1 x_1 + a_2 x_2] = u(n)(a_1 x_1 + a_2 x_2) = a_1 T[x_1] + a_2 T[x_2]$.

Variante: $y_k(n) = x(n - k)u(n)$ y $y(n - k) = x(n - k)u(n - k)$. Para $k = 1$, $n = 0$: $y_1(0) = x(-1)$ pero $y(-1) = x(-1)u(-1) = 0$, que difieren si $x(-1) \neq 0$.

Estático, causal: solo usa $x(n)$. Estable: $|y(n)| \leq |x(n)| \leq M_x$.

e) $y(n) = x(-n)$

Lineal: $T[a_1 x_1 + a_2 x_2](n) = a_1 x_1(-n) + a_2 x_2(-n)$.

Variante: la respuesta a $x(n - k)$ es $y_k(n) = x(-n - k)$, mientras que $y(n - k) = x(-(n - k)) = x(-n + k)$; difieren para $k \neq 0$.

Dinámico: $y(1) = x(-1)$. No causal: $y(-1) = x(1)$ depende de una muestra futura.

Estable: $|y(n)| = |x(-n)| \leq M_x$.

f) $y(n) = \max\{x(n), 0\}$

No lineal: con $x = 1$ y $a = -1$, $T[ax] = \max\{-1, 0\} = 0$, pero $aT[x] = -1 \cdot 1 = -1$; falla el escalado.

También falla la aditividad: $x_1 = 1$, $x_2 = -1$ dan $T[x_1 + x_2] = 0 \neq T[x_1] + T[x_2] = 1$.

Invariante: $y_k(n) = \max\{x(n - k), 0\} = y(n - k)$ (la operación n

- a) $y(n) = x(n - 2) + x(n + 1)$.
- b) $y(n) = x(2n)$.
- c) $y(n) = x(n)u(n)$, donde $u(n)$ es el escalón unitario.
- d) $y(n) = x(-n)$.
- e) $y(n) = \max\{x(n), 0\}$.